CM #6
Graphes

Introduction au projet

TI301 - P2 2025/2026

Nicolas FLASQUE

Graphes

Un graphe est un ensemble de **sommets** reliés par des **arêtes.**
(en anglais : sommet = vertex, arête = edge)

C'est une structure de données très riche :

- ✓ Étude théorique très poussée en mathématiques (théorie des graphes)
- ✓ De nombreux algorithmes très efficaces ont été développés
- ✓ Applications multiples : cartographie – Waze, RATP; systèmes critiques (nucléaire, aérospatial) : réseaux de Petri; gestion de workflow ou de processus

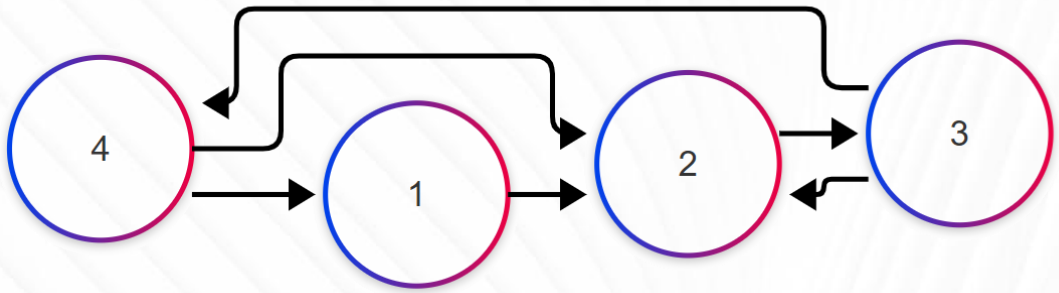
Une définition pratique

Définitions

Un **sommet** porte un numéro unique (un entier, de *1* à … *N*)

Une **arête** relie un sommet *i* à un sommet *j*, et est **orientée** :
« on peut aller du sommet *i* au sommet *j* », mais pas « de *j* à *i* ».
(il faudra une seconde arête dans ce cas).

Exemple



On peut aller de 1 à 2, mais pas de 2 à 1
On peut aller de 2 à 3 et de 3 à 2
On peut aller de 3 à 4
…

Représentations de graphe

Utiliser une structure de données :

Une « **matrice d'adjacence** »

Dans un graphe avec N sommets, on utilise une matrice M avec N lignes et N colonnes

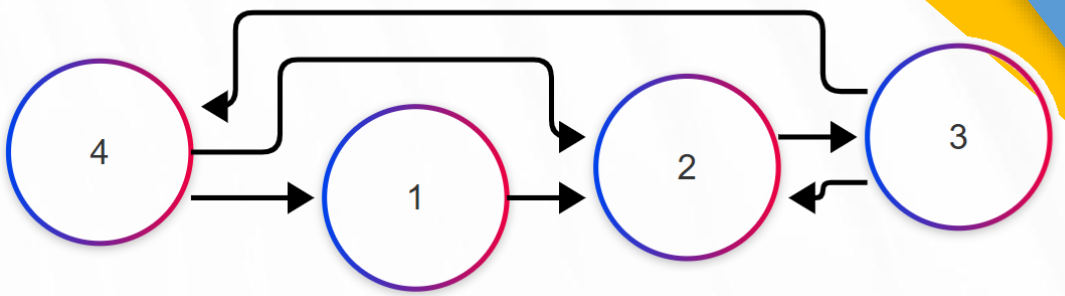
On utilise la règle :

si M_{ij} =1, il y a une arête du sommet i vers le sommet j

Si M_{ij} = 0, il n'y a pas d'arête du sommet i vers le sommet j

Exemple de matrice d'adjacence

Pour le graphe pris en exemple



La matrice *M* est

$$M=\begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix}$$

Il y a une arête de 1 vers 2
Il y a une arête de 2 vers 3
Il y a une arête de 3 vers 2, et une de 3 vers 4
Il y a une arête de 4 vers 1, et une de 4 vers 2

Représentations de graphe

Utiliser une structure de données :

Une « **liste d'adjacence**»

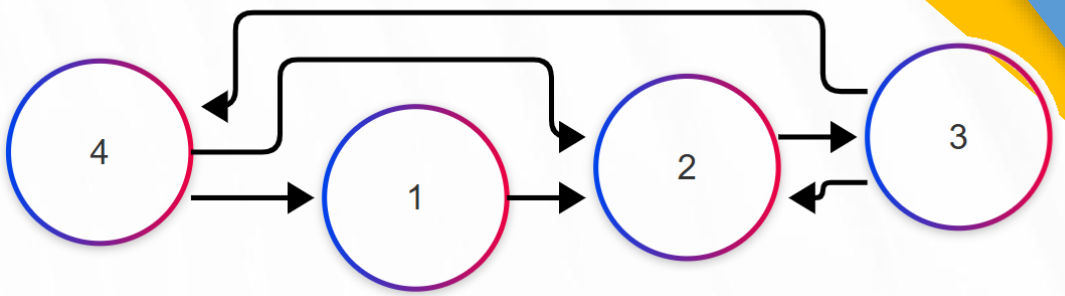
Dans un graphe avec *N* sommets, on a *N* liste chaînées :

Chaque liste stocke, pour un sommet *i*, toutes les arêtes 'sortantes' de *i* vers d'autres sommets;

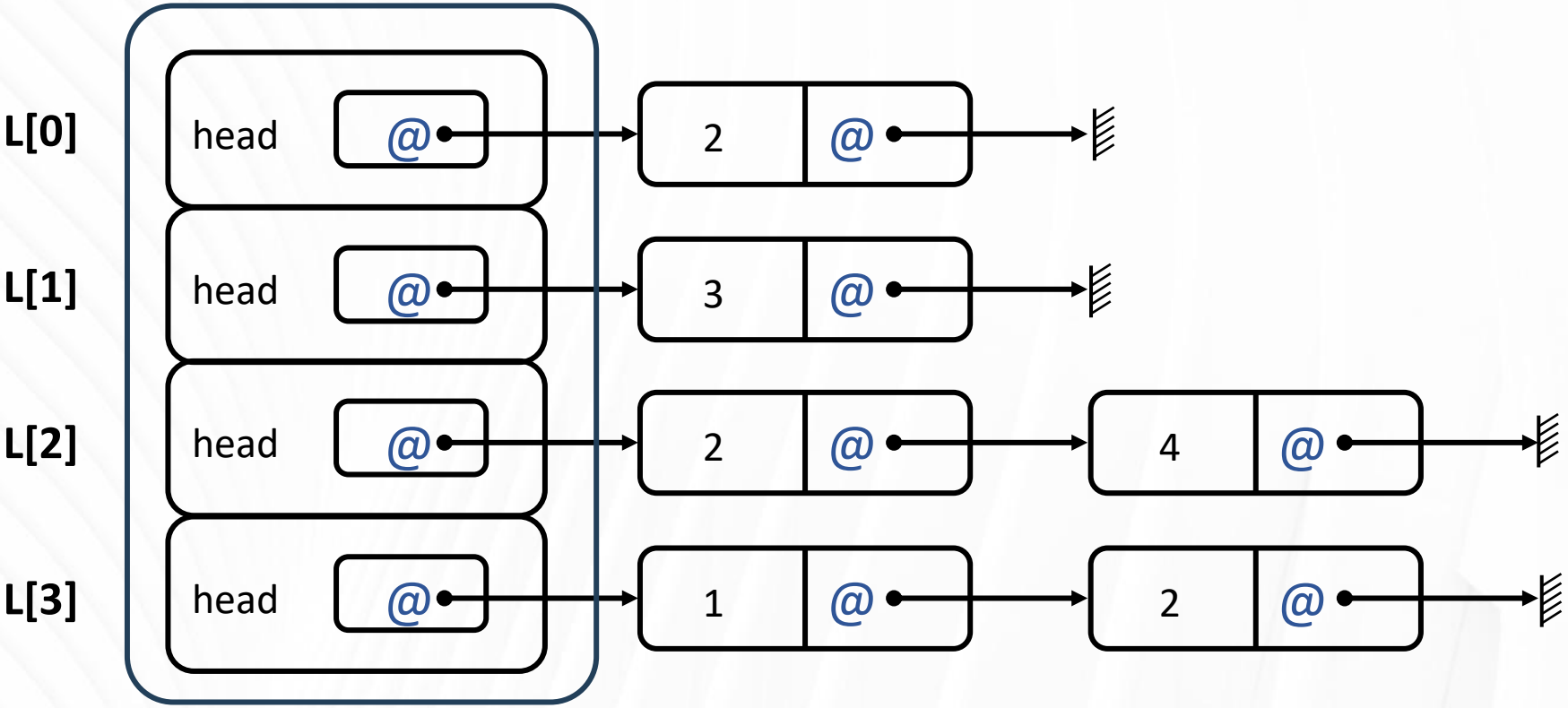
Les *N* listes chaînées sont stockées dans un tableau à *N* éléments (un élément pour chaque sommet).

Exemple de liste d'adjacence

Pour le graphe pris en exemple



La liste *L* est :



Représentations de graphe

Utiliser une structure de données :

Un « **arbre pour un graphe**»

Mais !

- Dans cet arbre, un nœud est un sommet…
- Il peut y avoir de 0 à N fils
- Et des liens qui ‘remontent’ dans l’arbre
- Algorithmes classiques d’arbres inefficaces

Alors, matrice d'adjacence ou liste d'adjacence ???

Question sous-jacente ; quelle est la « **meilleure** structure de données » ?

Réponse : « ça dépend »

Pour le projet :

il y a « peu » de liens d'un sommet vers un autre

On utilise des algorithmes pour 2 aspects :

- ✓ Théorie des graphes
- ✓ Application aux probabilités

Complexité

Hypothèse : dans un graphe, on aura au maximum **10** arêtes sortant d'un sommet :

Pour *N* sommets (*N* ≈ 1000)

Mémoire pour stocker les arêtes (une arête = 8 octets)

- ✓ **Matrice :** *NxN* éléments : 1000 x 1000 = 1 million → 8 m0
- ✓ **Liste** : au pire : 10 x 1000 : 80 kO

Choix retenus

Pour les parties 1 et 2 : seule l'existence des arêtes sera importante

→ **listes d'adjacence** pour les algorithmes

Pour la partie 3 : des valeurs seront associées aux arêtes pour les calculs

→ **Matrices d'adjacence** pour les algorithmes

Construire une liste d’adjacence

Une liste d’adjacence est un **tableau** (dynamique) dont chaque élément est une « **liste** »

Chaque liste stocke des « **cellules** »

Chaque « **cellule** » stocke des informations sur les arêtes du graphe

→ Vous savez faire chaque étape

Graphes utilisés pour le projet

Chaque arête d'un sommet *i* vers un sommet *j* aura un « poids »

Ce poids sera une valeur réelle **strictement positive** et **≤ 1**

Le type « **cellule** », utilisé dans les listes, stockera donc :

Le sommet de destination → un `int`

Le poids de l'arête → un `float`

Le type « **liste** » sera une liste simplement chaînée de « **cellules** », donc une structure avec un champ « `head` » pointant sur la première cellule

Données d'entrée

Pas de saisie utilisateur !

Pour un graphe avec *N* sommets, il faudrait :

- ✓ Saisir *N* pour le nombre de valeurs;
- ✓ Saisir *NxN* valeurs pour chaque arête.

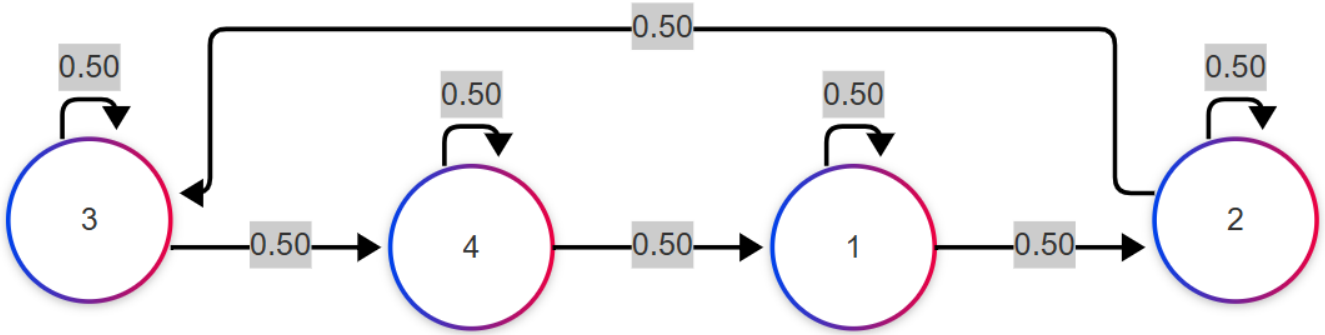
Nombre de sommets *N*	Saisies : N^2+1
4	17
10	101
100	10 001
1000	1 000 001

Lecture des fichiers de données

Fichiers de données fournis :

Ligne 1 : **`nombre_de_sommets`**(N)
Lignes suivantes :
`Sommet_de_départ sommet_d_arrivée  poids`

```
4
1 1 0.5
1 2 0.5
2 2 0.5
2 3 0.5
3 3 0.5
3 4 0.5
4 1 0.5
4 4 0.5
```



Construire la liste d'adjacence

À partir des données du fichier :

Lire la ligne 1 : on obtient le nombre de sommets *N*

→ Créer un tableau dynamique de *N* listes (vides)

Puis sur chaque ligne :
`sommet_de_départ sommet_d_arrivée  poids`

Ajouter à liste correspondant au **`sommet_de_départ`**
: une cellule avec **`sommet_d_arrivée  poids`**

Attention aux indices pour les listes ! **`L[0]`** pour le sommet *1*, **`L[i-1]`** pour le sommet *i* !

Lecture des fichiers de données

Dans le sujet de projet :

Vous aurez un code quasi-complet pour la lecture de fichier

Ce que vous devrez faire :

- ✓ Définir les structures de données « cellule », « liste » et « liste d'adjacence » → des types et des fonctions en langage C
- ✓ Adapter ce code pour ces types que vous aurez choisis

Technical corner

Quelques conseils pour réussir votre projet

En tout cas, pour vous éviter quelques écueils qui pourraient vous bloquer !

1) Système de fichier de projet
2) Fichier **`CMakeLists.txt`**
3) Utilisation des fonctions **`sprintf()`**, **`strcpy()`** et **`strcat()`**

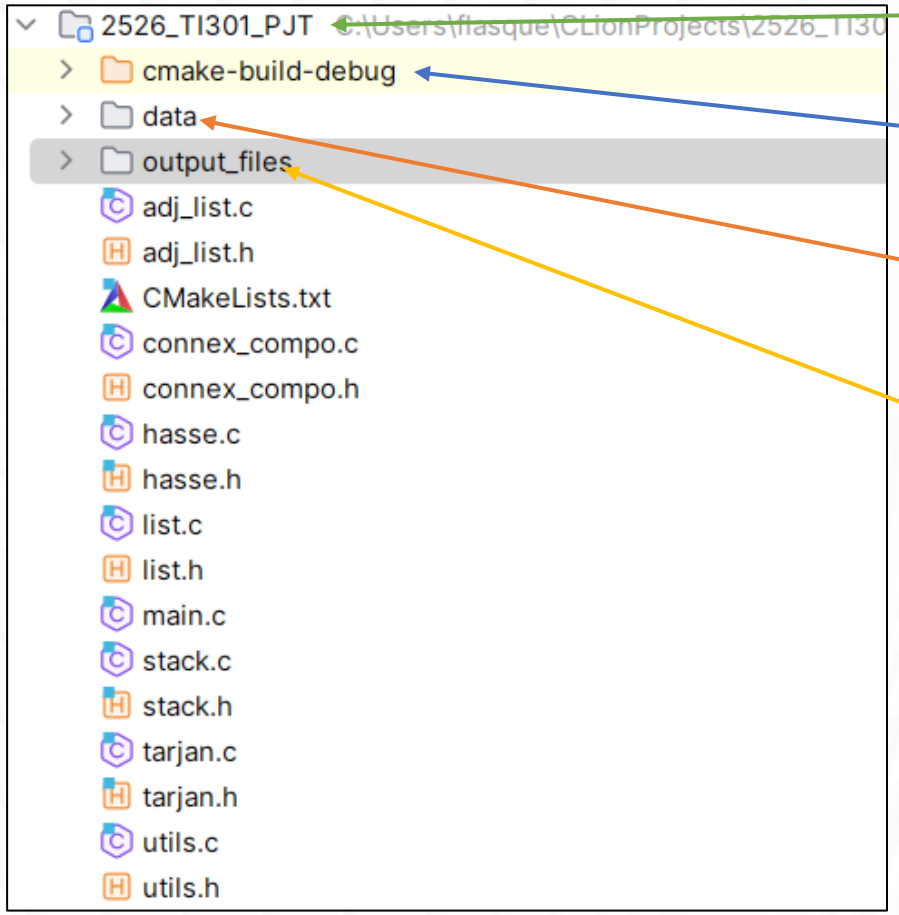
Système de fichier du projet

À partir d'un projet créé avec CLion

- ✓ *Le répertoire 'par défaut'* est celui dans lequel se trouvent vos fichiers source
- ✓ Le répertoire dans lequel se trouve le fichier exécutable se nomme **`cmake-build-debug`**
- ✓ D'autres fichiers (notamment les fichiers de données) peuvent se trouver dans un répertoire (par exemple un répertoire **`data/`** , sous-répertoire de celui 'par défaut')

Système de fichier du projet

Illustration avec un exemple



Répertoire ‘par défaut’

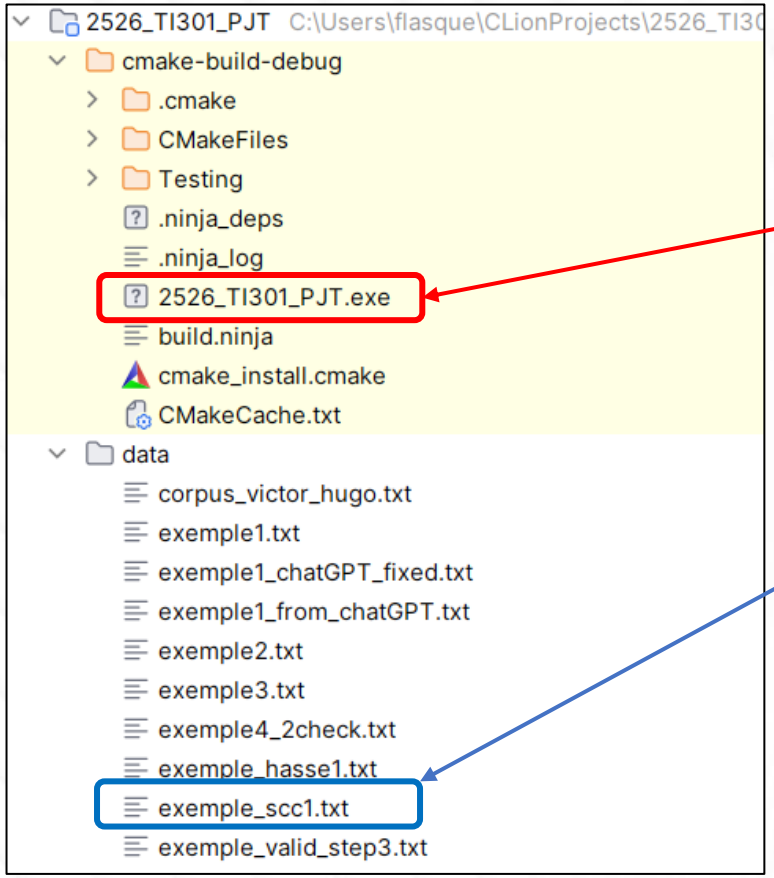
Répertoire où se trouve l’exécutable

Répertoire des fichiers de données

Répertoire où écrire vos fichiers ‘résultats’

Accès aux fichiers depuis l'exécutable

Pour accéder à un fichier de données depuis l'exécutable :
Il faut préciser le chemin d'accès



Pour accéder au fichier '**`exemple_scc1.txt`**' depuis l'exécutable :

Remonter d'un répertoire pour revenir au répertoire par défaut

Puis aller dans le répertoire 'data'

`"../data/exemple_scc1.txt"`

Fichier CMakeLists.txt

Ce fichier indique comment construire une ‘cible’ à partir de ‘sources’

```
cmake_minimum_required(VERSION 3.27)
project(2526_TI301_PJT C)

set(CMAKE_C_STANDARD 11)

add_executable(2526_TI301_PJT
    main.c adj_list.c utils.c connex_compo.c
    tarjan.c stack.c list.c hasse.c)
```

Pour construire l’exécutable (add_executable) nommé **2526_TI301_PJT**

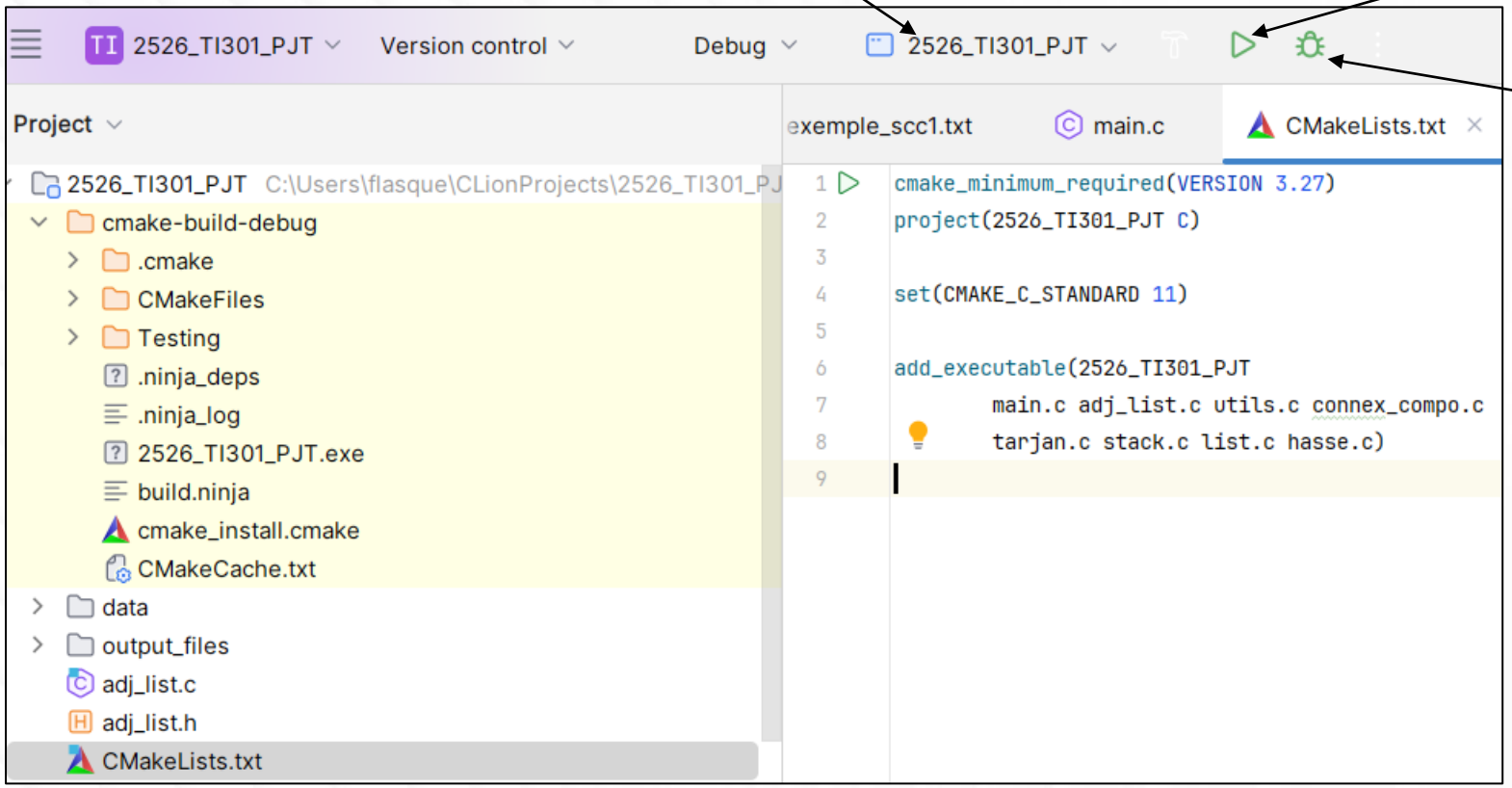
Il faut utiliser les fichiers source

On peut : choisir/changer le nom de l’exécutable

Seule obligation : il faut une fonction **`main()`** dans un des fichiers, et une seule

Lien avec l'interface

La 'cible' – l'exécutable – apparaît dans l'interface CLion



Le bouton 'Run' lance la compilation et l'exécution (si tout va bien)

Le bouton 'Debug' permet de poser des 'points d'arrêt' pour exécuter le programme pas à pas

Cibles multiples

Dans un projet Clion, on peut indiquer plusieurs ‘cibles’, plusieurs exécutables (par exemple, un exécutable pour tester un sous-ensemble de fonctions) à construire.

par exemple, on veut :

Un exécutable pour tout le projet

Un exécutable pour tester le chargement d’un fichier

Mais on veut éviter de changer la fonction **`main()`** à chaque fois, de mettre des parties de **`main()`** en commentaire….

Cibles multiples

On va donc utiliser un fichier .c (on choisit le nom qu'on veut, par exemple **`toto.c`**, et l'associer à une nouvelle cible, un nouvel exécutable)

Pour ce faire, on édite le fichier CMakeLists.txt :

Exemple :

```
cmake_minimum_required(VERSION 3.27)
project(2526_TI301_PJT C)

set(CMAKE_C_STANDARD 11)

add_executable(TI_301_PJT
        main.c adj_list.c utils.c connex_compo.c
        tarjan.c stack.c list.c hasse.c)

add_executable(TEST_LOAD_FILE
        toto.c adj_list.c utils.c)
```

On ajoute un nouvel exécutable nommé **`TEST_LOAD_FILE`**, construit avec les fichiers source :

`toto.c`
`adj_list.c`
`utils.c`

Liens avec l'interface

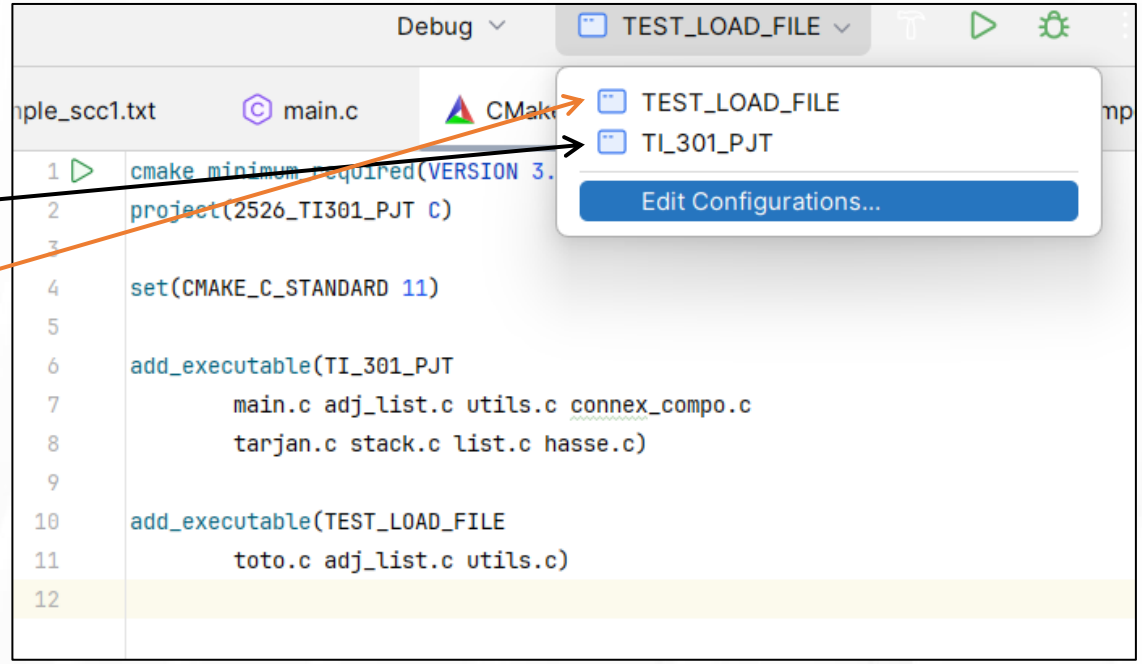
On peut donc mettre un nouveau **`main()`** dans le fichier **`toto.c`** et faire des tests, sans modifier l'autre !

```
cmake_minimum_required(VERSION 3.27)
project(2526_TI301_PJT C)

set(CMAKE_C_STANDARD 11)

add_executable(TI_301_PJT
        main.c adj_list.c utils.c connex_compo.c
        tarjan.c stack.c list.c hasse.c)

add_executable(TEST_LOAD_FILE
        toto.c adj_list.c utils.c)
```



Pour avancer sereinement

Vous pouvez ajouter autant de cibles (exécutables) que vous voulez, notamment pour tester vos fonctions ‘individuellement’

Conseils :

Procédez par étapes et faites autant d’exécutables que nécessaires

N’ajoutez pas les fichiers .h dans CMakeLists : ils devraient être inclus correctement dans les fichiers .c qui seront compilés

Utiliser sprintf, strcat et strcpy

Dans le projet, il vous sera demandé de créer des fichiers texte (on vous laisse la surprise).

Pour écrire dans un fichier texte, on utilise le plus souvent :

```
FILE *fichier;
fichier = fopen(nom_du_fichier, "wt");
```

Où le nom du fichier est un `char *`, et `"wt"` indique 'Write Text'

Pour écrire dans ce fichier, on utilise le plus souvent la fonction `fprintf` ('file'printf pour 'printf vers un fichier')

Utiliser sprintf, strcat et strcpy

Il est parfois utile de procéder de la manière suivante :

Plutôt que d'écrire les données 'à la volée' dans un fichier, on va les 'préparer' dans une chaîne de caractères, puis écrire cette chaîne en une seule fois dans le fichier.

Cette technique est similaire à celle que l'on utilise volontiers en Python : on construit une chaîne, puis on la 'sauvegarde' en une seule fois dans le fichier

Quels sont les avantages ?

Un cas d'utilisation

Vous devez écrire un fichier texte pour un graphe, en deux parties :

Des informations sur les sommets (en un bloc de texte)
Puis
Des informations sur les arêtes (en un autre bloc de texte)

Mais, pour chaque sommet, vous pouvez obtenir : l'information sur le sommet , puis celle sur les arêtes

Avec **`fprintf`** seul, vous devez parcourir la structure deux fois :

Une fois pour le bloc de texte 'sommets'
Une fois pour le bloc pour le bloc de texte 'arêtes'

Utiliser sprintf()

On va donc utiliser une technique similaire à celle utilisée en Python, mais avec les fonctions de base du Langage C

Dans une fonction, on crée des chaines de caractères 'vides' suffisamment grandes, une pour chaque 'bloc' à écrire dans le fichier

```
char bloc1[10000]; // détruite à la fin de la fonction
donc pas de problème de mémoire à long terme
char bloc2[10000]; // idem - 10000 char c'est 10 kO
```

On utilise `sprintf` ('string printf' pour 'printf dans une chaîne')

Utiliser sprintf()

`sprintf` s'utilise comme `printf`, on doit juste mettre en premier paramètre, la chaine dans laquelle on veut 'printf'

Exemple :

```
char bloc1[10000];
sprintf(bloc1, "sommet %d :",1);
// ceci 'affiche' dans la chaîne 'bloc1' au lieu
d'afficher à l'écran
```

THE END

Le reste du cours est consacré à vos questions